Proyecto

Intermodular ASIR

Carlos Merino, Saskia Espinoza, Yamil Motta

15 de marzo del 2026

Esquema de red de Sede Pseudolocal (Cuenta AWS 1)..... 2
Esquema de red de Sede Nube (Cuenta AWS 2)..... 3
Esquema de red local de casa:..... 4
Esquema de red general..... 4
Arquitectura y Desarrollo de la Aplicación Web (Portal IT)..... 5
Planificación semanal..... 7

Esquema de red de Sede Pseudolocal (Cuenta AWS 1)

En esta red se aloja el AD para la gestión de credenciales centralizada y roles para Jenkins.

En principio, una cuenta con una VPC propia que contiene:

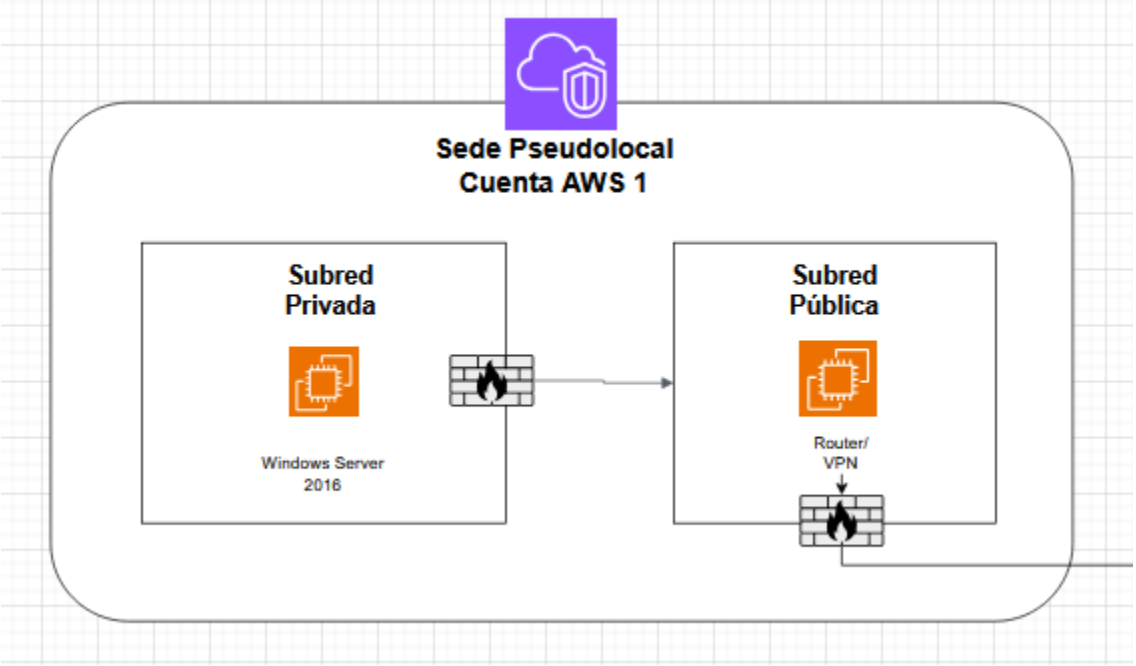
Subred privada:

- Instancia EC2 con los siguientes servicios:

  Windows Server 2016, con el servicio de Active Directory

Subred publica:

- Instancia con VPN que actúa también como NAT Gateway.



Esquema de red de Sede Nube (Cuenta AWS 2)

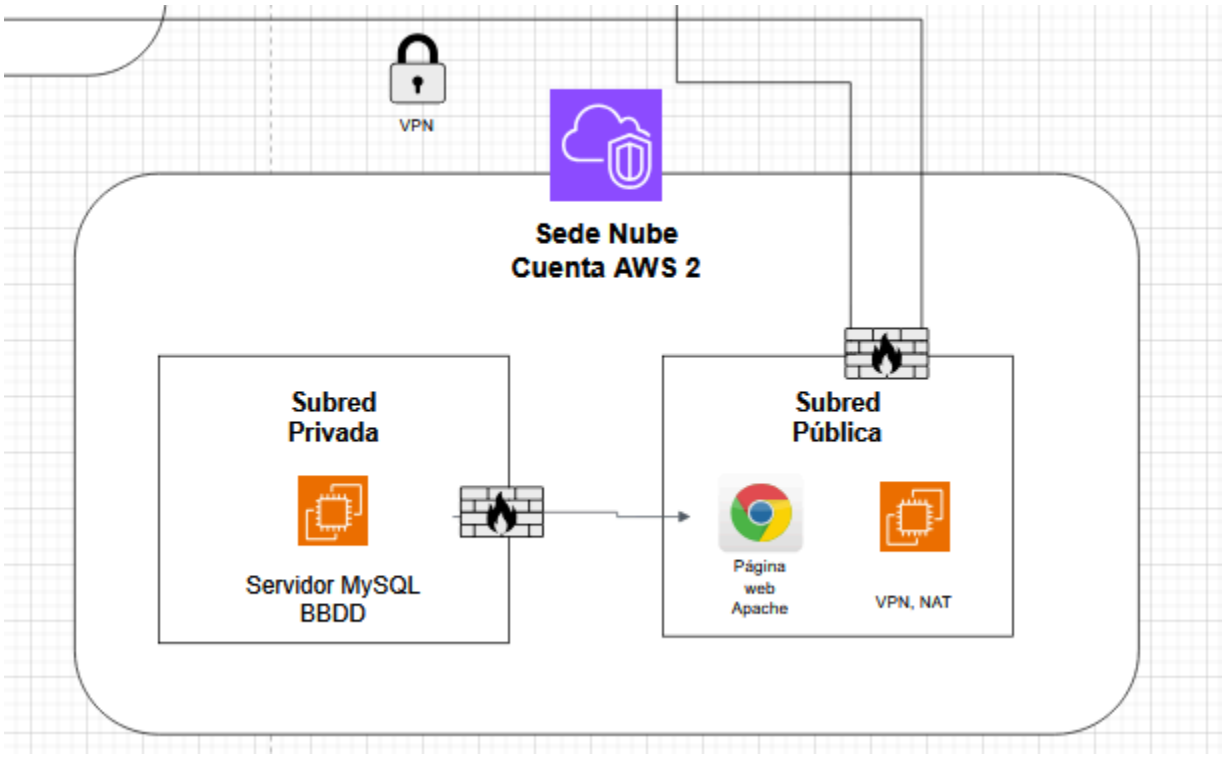
Esta red, “Sede Nube” alojaría la página web Apache como tal, junto a su servidor de base de datos MySQL en una subred privada (Backend).

En Subred privada:

- El Servidor MySQL que contiene datos sobre la página web Apache.

En subred pública:

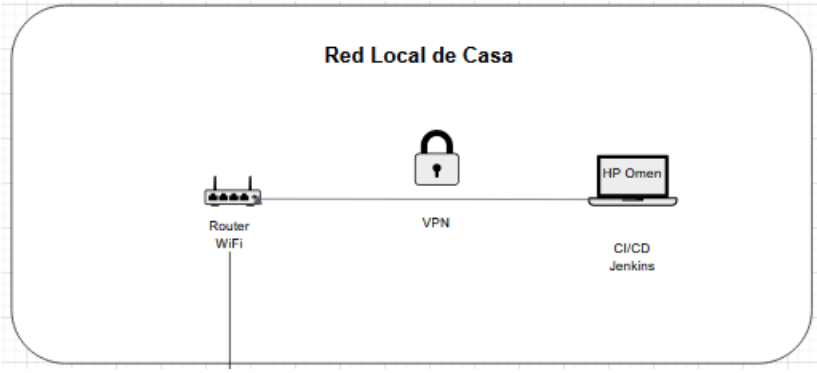
- La página web Apache (aplicación web).
- Instancia de VPN/NAT para conectar con sede Pseudolocal y Red Local del CI/CD.



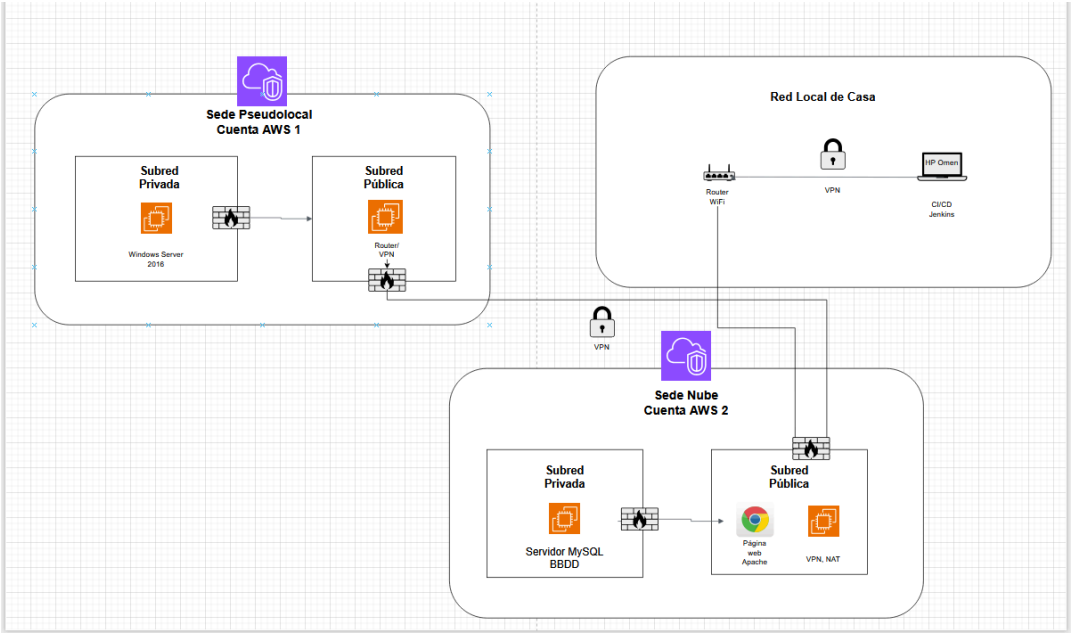
Esquema de red local de casa:

Esta red está hecha para no agotar los recursos de AWS, consumiría muchos créditos una instancia para Jenkins y pretender que llegue hasta junio.

- Motor CI/CD (Virtualización Local)**:** Se emplea una máquina virtual dedicada a ejecutar Jenkins y Docker. Al estar en local, este entorno dispone de la potencia de CPU y memoria RAM necesarias para compilar el código y ejecutar las auditorías de seguridad.



Esquema de red general



Aplicación Web (Portal IT)

Este bloque es la página web que desarrollamos para el proyecto. En realidad, la web no es el producto final en sí, sino nuestra herramienta de pruebas. Es un gestor de incidencias muy sencillito que nos sirve de excusa perfecta para comprobar que toda la red, la seguridad y la automatización que hemos montado por detrás funciona de verdad.

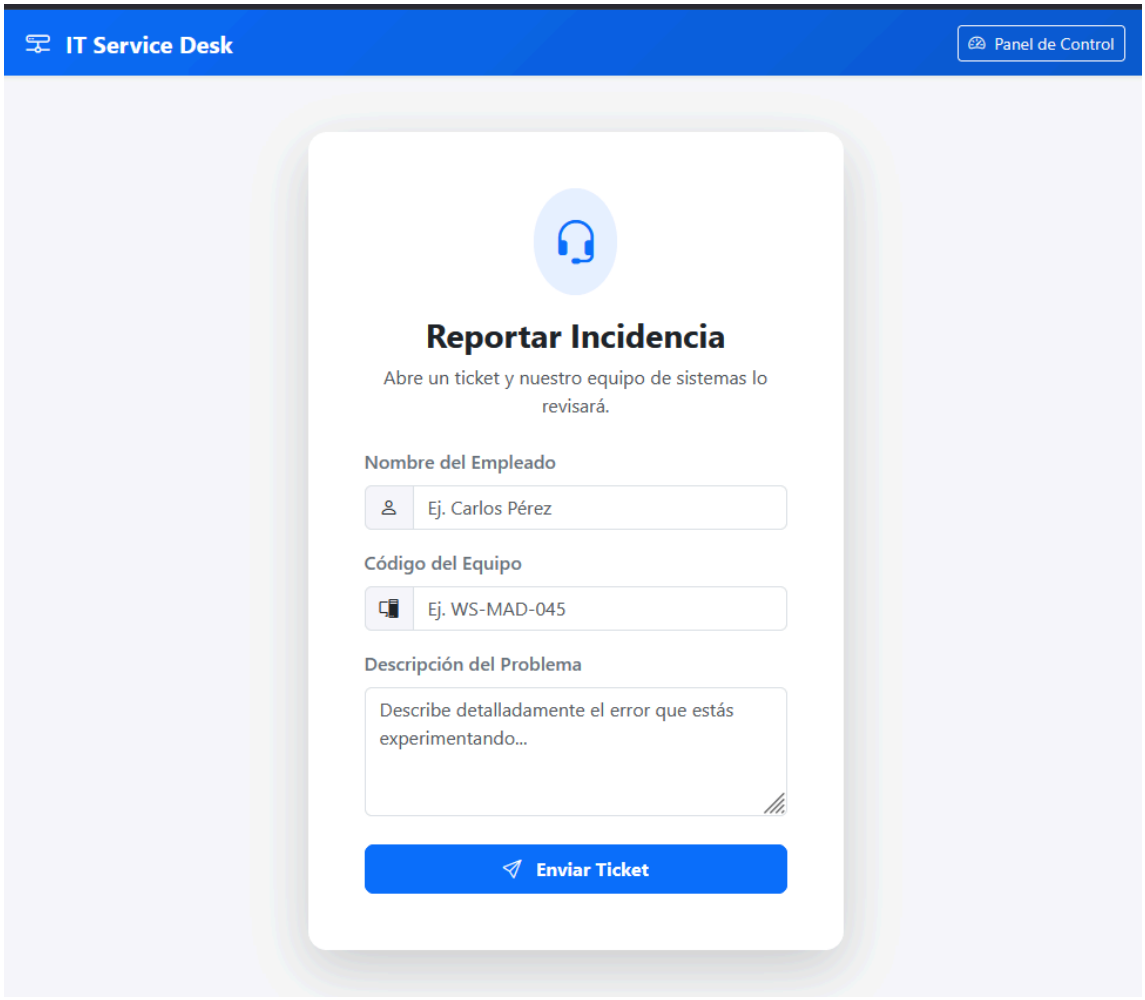
- El porqué de la web: Queríamos hacer algo realista, parecido a lo que usa cualquier empresa para que los empleados reporten fallos informáticos. Al hacer que la web (que está abierta en la red pública) tenga que guardar los datos en un servidor MySQL (que está escondido en la red privada), demostramos en directo que nuestra VPN y nuestras reglas de red se comunican perfectamente.

- Cómo lo pensamos (Requisitos): PHP y MySQL. Descartamos usar frameworks pesados porque las máquinas de AWS del Learner Lab tienen muy poca memoria y no queríamos que el servidor sufriera. A nivel técnico, el reto más importante era no dejar la contraseña de la base de datos escrita en el código. Lo preparamos para que la web lea las claves desde las variables del sistema; así, Jenkins se las puede inyectar de forma segura justo en el momento de subirla a producción.

- El diseño visual (UI/UX): Tiene dos pantallas muy directas: un formulario para que el usuario envíe el ticket y un panel de control para que nosotros veamos la lista. Para que se viera profesional sin tener que pasarnos horas picando código CSS a mano, le metimos Bootstrap 5. Queda corporativo, limpio y funciona bien en móviles. Además, le añadimos un pequeño script en JavaScript al panel para poder buscar tickets al vuelo sin tener que recargar la página.

- El empaquetado con Docker: El código de la página son apenas tres archivos PHP. Para quitarnos de encima el típico problema de "pues en mi ordenador sí que funcionaba", metimos todo ese código y el servidor Apache dentro de un contenedor usando un archivo Dockerfile. De esta forma, la web viaja en un paquete cerrado y listo para que nuestro sistema la despliegue en AWS sin fallos.

- El cebo de seguridad: Este es el detalle más divertido de la web. En el código del formulario dejamos un fallo de seguridad a propósito (una inyección SQL). Lo hicimos para poder demostrar que las herramientas de seguridad que configuraremos en el proyecto hacen su trabajo. Así, cuando intentemos subir este código, el sistema lo revisará, detectará el agujero de seguridad y bloqueará la subida antes de que llegue a producción.


IT Service Desk
Panel de Control
Reportar Incidencia
Abre un ticket y nuestro equipo de sistemas lo revisará.
Nombre del Empleado
Ej. Carlos Pérez
Código del Equipo
Ej. WS-MAD-045
Descripción del Problema
Describe detalladamente el error que estás experimentando...
Enviar Ticket

Planificación semanal

Disclaimer: Esta es una planificación muy general y puede ir cambiando a medida que avancemos con el proyecto.

Semanas 1 y 2: Preparar las máquinas

Aquí básicamente vamos a montar la estructura vacía. En AWS creamos la red (con su parte pública y privada) y dejamos creados los servidores Linux básicos para la web, la base de datos y el router. En el VMware de casa hacemos lo mismo: instalamos las máquinas virtuales que luego usaremos para el Windows Server y el Jenkins. De momento solo instalamos los sistemas operativos, sin configurar nada.

Semanas 3, 4 y 5: Unir las redes (VPN)

Esta es la parte más delicada, por eso nos damos tres semanas. Tenemos que conectar el VMware local con la nube de Amazon usando WireGuard. El gran problema es que AWS nos apaga las máquinas cada 4 horas y nos cambia la IP pública. Para no tener que reconfigurar todo a mano cada vez, vamos a hacer unos scripts con DuckDNS para que el túnel VPN se levante y se reconecte solo al arrancar. Cuando consigamos hacer ping desde casa hasta la nube, pasamos a lo siguiente.

Semanas 6 y 7: Levantar los servicios

Con la red ya unida y funcionando, instalamos los programas. En la parte pseudolocal convertimos el Windows en Controlador de Dominio (Active Directory) y creamos los usuarios de prueba. En la parte de AWS, instalamos el MySQL en la red privada (cerrado por firewall para que solo se pueda entrar por la VPN) y dejamos listo el servidor web en la red pública.

Semanas 8, 9 y 10: Automatizar los despliegues

Aquí montamos el motor para no tener que subir las actualizaciones a mano. Instalamos Docker y Jenkins en la máquina de casa y lo conectamos a nuestro GitHub. La idea es que, en cuanto hagamos un cambio en la página web, Jenkins lo pille, lo empaquete y lo mande directo al servidor de Amazon a través de la VPN.

Semanas 11 y 12: Seguridad y filtros

Para que el proyecto tenga nivel, el código tiene que pasar unos controles antes de ir a producción. Vamos a meter herramientas como Trivy para escanear los contenedores y SonarQube (o algo más ligero si nos quedamos sin RAM) para buscar fallos de programación.

También vamos a configurar Jenkins para que las contraseñas de la base de datos estén ocultas y no queden escritas a la vista en el código fuente.

Semana 13: Pruebas finales y memoria

La última semana es para comprobar que todo aguanta. Vamos a apagar AWS a lo bestia simulando un corte para ver si los scripts levantan la red solos al reiniciar. Haremos un cambio real en el código para ver si todo el proceso automático funciona del tirón. Sacamos capturas de todo esto, redactamos las conclusiones y cerramos el documento para la presentación.